

Full-Stack Development Training: Checkpoint 2

Time Allowed: 4 Hours | Total Marks: 100

SECTION 1: SETUP AND INSTRUCTIONS

- You will be given a folder containing mock-tasks.json (sample data) and an empty project scaffold. Do not connect to the internet at any point - npm install will not work and is not needed; everything required (Jest, a static file server if you want one) is already installed.
- Plain HTML/CSS/JavaScript only. No frameworks (React/Vue/etc.), no build tools, no external CDN scripts.

SECTION 2: PROJECT BRIEF

PodBoard is a lightweight task tracker for a training pod. Build a single-page app with three views and no page reloads:

- **Board view** (default) - three columns: *To Do*, *In Progress*, *Done*. Each task is a card showing title, assignee, and priority (low/medium/high).
- **Archive view** - a read-only list of tasks that have been deleted from the board (soft-delete, not gone forever).
- **Stats view** - a small read-only summary: total tasks, count per column, count per priority.

Navigate between the three views using the browser URL (e.g. `#/board`, `#/archive`, `#/stats`) - back/forward buttons must work correctly, and reloading the page on any view must land you back on that same view.

SECTION 3: FUNCTIONAL REQUIREMENTS

- **Load initial data** from mock-tasks.json using an async function that *simulates* a network call (wrap the file read/fetch in a setTimeout-based delay of ~400ms) so you have a genuine loading state to build against - do not call this synchronously.
- **Add a task**: a form with title (required, 3–60 chars), assignee (required), priority (defaults to medium). Reject invalid input with a visible, specific error message next to the offending field - do not use `alert()`.
- **Move a task** between columns. At minimum, provide button controls (e.g. “→ In Progress”) that work with keyboard alone - drag-and-drop is optional bonus, not a substitute for the keyboard-accessible path.
- **Delete a task** from the board - moves it to Archive, it does not disappear from the data entirely.
- **Restore a task** from Archive back to *To Do*.
- **Persist all state to localStorage** - a full page reload must restore the exact same board/archive state, not re-fetch mock-tasks.json from scratch. (Only load mock-tasks.json the very first time there is nothing in localStorage yet.)
- **Loading and error states**: while the initial mock fetch is in flight, show a visible loading indicator; if you simulate a failure (see below), show a visible, specific error state - not a blank screen or a raw console error.
- **Accessibility basics**: every interactive control must be reachable and operable by keyboard alone (Tab, Enter/Space); use correct semantic elements (`<button>`, not clickable `<div>`s); any status change relevant to screen reader users (e.g., “task moved”) should be reflected in the DOM in a reasonable way - you do not need a full ARIA live-region implementation, but you should be able to explain in your viva what you’d add given more time.

SECTION 4: TECHNICAL STANDARDS

- **State management:** implement your own small store - something with the shape of `getState()`, `dispatch(action)`, `subscribe(listener)` - that holds the single source of truth for tasks. Do not scatter task state across multiple ad-hoc global variables. Components/render functions should re-render in response to state changes via subscription, not by being called manually from inside every event handler.
- **Use at least one design pattern deliberately** (Observer via your store's `subscribe`, Module pattern to encapsulate the store's internals, or a Factory function to construct task objects with defaults/validation) and be ready to name it and justify the choice in your viva.
- **Error handling:** define at least one custom error class (extending `Error`) for validation failures, and handle it distinctly from unexpected errors.
- **Code organization:** split your JavaScript into at least three files/modules using ES modules (`import/export`) - e.g. `store.js`, `router.js`, `render.js`, `main.js`. A single 400-line `script.js` will be marked down under code quality even if functionally complete.
- Write a Jest test file (`store.test.js` or similar) with **at least 5 tests** covering your store's pure logic - dispatching actions and asserting the resulting state - without touching the DOM. Jest is pre-installed; run with `npx jest`.
- You do not need to test DOM rendering for this checkpoint; keep your store logic testable in isolation from the DOM specifically so this is possible without jsdom setup work eating your time.

SECTION 5: DELIVERABLES

A single project folder containing your HTML/CSS/JS files, `mock-tasks.json` (unchanged or extended, your choice), your Jest test file, and a short `README.md` (10–15 lines) explaining: which design pattern(s) you used and why, one thing you'd improve with more time, and how to run your tests. Zip the folder and submit through the usual channel before time is called - work saved after the deadline is not graded.

SECTION 6: SUGGESTED TIME ALLOCATION

Phase	Suggested time
Read brief, plan module structure on paper, set up file skeleton	20 min
Store (state management) + task Factory/validation + custom error class	45 min
Router (hash-based navigation across the 3 views)	30 min
Board view: render, add/move/delete/restore, keyboard accessibility	70 min
Mock async load with loading/error states, localStorage persistence	30 min
Archive + Stats views	25 min
Jest tests for store logic	30 min
README, final polish, zip & submit	20 min